

第二章 创建数据集

一、理解数据集

- 数据集通常是由数据构成的一个矩形数组，**行表示观测值，列表示变量**
- R可以处理的数据类型包括数值型、字符型、逻辑型、复数型和原生型

二、数据结构

1、向量：是用于储存数值型、字符型或逻辑型数据的一堆数组。函数c()创建向量

表现形式

- 注意：**标量**是指含一个元素的向量，例如，`f<-3`，它们通常用于保存常量
- 数值型
 - `a<-c(9.1,8.2,7.3)`
 - `b<-1:10`,
- 字符型
 - `char<-c('a','b')`，**要加单引号或双引号**
- 逻辑型
 - `logit<-c(TRUE,FALSE,T,F)`
- 混合类型
 - `c(1, "two", TRUE)`
 - R 会自动强制转换，通常转成字符型

访问形式

- 通过在**方括号**中给定元素所处位置的数值，可以访问向量中的元素
 - `a[c(1,3)]`，表示访问向量a中的第1个和第3个元素
 - `a<-c(2:6)`等价于`a<-c(2,3,4,5,6)`
 - `a[-2]`，表示去掉第 2 个元素

易错点

- R 的索引从 1 开始，不是从0开始
- `a[2:6]` 和 `a[c(2, 6)]` 不一样
 - `a[2:6]` 是第 2 到第 6
 - `a[c(2, 6)]` 只取第 2 和第 6

2、矩阵：一个二维数组，只是每个元素都拥有相同的模式（数值型、字符型或逻辑型），函数matrix()创建矩阵

表现形式：

`mymatrix<-`

`matrix(vector,nrow,ncol,byrow=logical_value,dimnames=list(rnames,cnames))`

- vector包含了矩阵的元素
- nrow和ncol表示行和列
- dimnames包含了可选的、以字符型向量表示的行名和列名
- 选项byrow表明矩阵应当按行填充（byrow=TRUE）还是按列填充（by=FALSE）

代码创建示例

- 创建一个5×4的矩阵
 - `y <- matrix(1:20, nrow = 5, ncol = 4)`
- 说明
 - matrix() 用于创建矩阵
 - 1:20 是填入的数据，R 默认按列填充矩阵，而不是按行填充
 - nrow = 5：5 行
 - ncol = 4：4 列

重要部分

创建数据、行名和列名

```
cells <- c(1, 26, 24, 68)
```

```
rnames <- c("R1", "R2")
```

```
cnames <- c("C1", "C2")
```

按行填充矩阵

```
mymatrix <- matrix(cells,  
  nrow = 2, ncol = 2, byrow = TRUE,  
  dimnames = list(rnames, cnames))  
mymatrix
```

按列填充矩阵

```
mymatrix <- matrix(cells,  
  nrow = 2, ncol = 2, byrow = FALSE,  
  dimnames = list(rnames, cnames))  
mymatrix
```

矩阵特点

- 二维
- 同类型
- 有行名和列名

扩展：查看矩阵属性

- `dim(y)`
- `nrow(y)`
- `ncol(y)`
- `rownames(mymatrix)`
- `colnames(mymatrix)`

3、数组：与矩阵类似，但是维度可以大于2，函数`array()`创建数组

表现形式

`myarray<-array(vector,dimensions,dimnames)`

- `vector`包含数组中的数据
- `dimensions`是一个数值型向量，给出了各个维度下标的最大值
- `dimnames`是可选的、各维度名称标签的列表

代码创建示例

```
dim1 <- c("A1", "A2")
dim2 <- c("B1", "B2", "B3")
dim3 <- c("C1", "C2", "C3", "C4")
z <- array(1:24, c(2, 3, 4), dimnames = list(dim1, dim2, dim3))
z
```

代码讲解

- 先定义三个维度的名字，分别是`dim1`、`dim2`、`dim3`
- 三个维度的长度分别是2、3、4，然后再创建数组
- `1:24`是数据，`c(2, 3, 4)`是维度大小，`dimnames`为每个维度设置名称，最后创建一个 $2 \times 3 \times 4$ 的三维数组

数组特点

- 多维
- 同类型

数组索引

- `z[1, 2, 3]`，取三维中的一个具体元素
- `z[, 1]`，取第一个层面
- `z[, 2]`，取第二个层面

4、数据框：函数`data.frame()`创建数据框

表现形式

`mydate <- data.frame(co12, co12, co13, ...)`

- 其中，列向量`co12`等可以为任何类型的数据
- 每一列的名称可以由函数`names`指定

代码创建示例

```
patientID <- c(1, 2, 3, 4)
age <- c(25, 34, 28, 52)
diabetes <- c("Type1", "Type2", "Type1", "Type1")
status <- c("Poor", "Improved", "Excellent", "Poor")
patientdata <- data.frame(patientID, age, diabetes, status)
patientdata
```

代码讲解

- 先分别创建`patientID`（数值）、`age`（数值）、`diabetes`（字符）和`status`（字符）四列数据
- 组合成数据框`patientdata`
- `data.frame()` 把多个向量按列组合，**每列长度必须相同，每列可以是不同类型**

选取数据框中的元素

- `patientdata[1:2]`，表示取前两列
- `patientdata$age`，表示取`age`这一列
- `patientdata[c("diabetes", "status")]`，取 `diabetes` 和 `status` 两列

扩展：查看数据框结构

- `class(patientdata)`
- `str(patientdata)`
- `summary(patientdata)`
- `dim(patientdata)`
- `names(patientdata)`

5、因子：分类变量（名义变量）和有序变量（顺序变量）在R中称为因子

代码

```
patientID <- c(1, 2, 3, 4)
age <- c(25, 34, 28, 52)
diabetes <- c("Type1", "Type2", "Type1", "Type1")
status <- c("Poor", "Improved", "Excellent", "Poor")
diabetes <- factor(diabetes)
status <- factor(status, order = TRUE)
patientdata <- data.frame(patientID, age, diabetes, status)
str(patientdata)
summary(patientdata)
```

代码讲解

函数`factor()`以一个整数向量的形式储存类别值，整数的取值范围是`[1...k]`（其中`k`是名义变量中唯一值的个数）

- `diabetes <- factor(diabetes)`
 - 原来 `diabetes` 是字符向量
 - 转换后变成因子
 - 因子用于表示分类变量
- `status <- factor(status, order = TRUE)`
 - `status` 也被转成因子
 - `order = TRUE` 表示这是有序因子
- `patientdata <- data.frame(patientID, age, diabetes, status)`
 - 表示重新创建数据框，此时数据框的列类型变成，`patientID`：数值型，`age`：数值型，`diabetes`：因子，`status`：有序因子
- `str(patientdata)`
 - 会显示对象类型、每列类型、因子的水平数
- `summary(patientdata)`
 - `summary()` 对不同类型的变量会显示不同信息

6、列表：一些对象的有序集合，函数`list()`创建列表

代码

```
g <- "My First List"
h <- c(25, 26, 18, 39)
j <- matrix(1:10, nrow = 5)
k <- c("one", "two", "three")
mylist <- list(title = g, ages = h, j, k)
mylist
```

```
mylist[[2]]  
mylist[["ages"]]
```

代码讲解

- g、h、j、k分别为创建的不同类型的对象
- g：字符串，h：数值向量，j：矩阵，k：字符向量
- list()用于创建列表，列表可以存放不同类型、不同长度、不同结构的对象
- mylist[[]], []用于提取列表中的单个元素，例如中间填写2，表示第2个元素是 ages，结果是25 26 18 39，2等同于ages

特点

- 列表是R中最灵活的数据结构，某个列表可能包含若干向量、矩阵、数据框等

7、tibble数据框：函数tibble()或as_tibble()来创建

创建tibble数据框

```
library(tibble)  
mtcars<-as_tibble(mtcars)  
mtcars
```

特点

- 不会更改变量的名称
- 对tibble数据框取子集总是返回一个tibble数据框
- tibble数据框不支持行名

三、数据的输入

1、使用键盘输入数据（最简单）

常见的两种方法

用R内置的文本编辑器

- 函数edit()会自动调用一个允许手动输入数据的文本编辑器
- 针对这个数据对象调用文本编辑器，输入数据，并将结果保存回此数据对象中

```
mydata <- data.frame(age = numeric(0), gender = character(0))  
mydata <- edit(mydata)
```

直接在代码中嵌入数据

- 使用函数`read.table()`处理字符串并返回数据框

2、从带分隔符的文本文件导入数据

- 使用`read.table()`从带分隔符的文本文件中导入数据。此函数可读入一个表格格式的文件并将其保存为一个数据框。表格的每一行分别出现在文件中的每一行
- 语法如下：

```
mydataframe<-read.table(file,options)
```

代码

```
grades<-read.table("studentgrades.csv",header=TRUE,  
row.names="StudentID",sep=",")  
str(grades)
```

说明：`header = TRUE` 表示第一行为列名

3、导入Excel数据

使用readxl包

- 安装：`install.packages("readxl")`
- 加载包：`library(readxl)`
- 读取：`myexcel <- read_excel("data.xlsx")`
- 按名称读取：`myexcel <- read_excel("data.xlsx", sheet = "Sheet1")`
- 按序号读取：`myexcel <- read_excel("data.xlsx", sheet = 1)`
- 读取指定区域：`myexcel <- read_excel("data.xlsx", range = "A1:C10")`

其他

- `xlsx`、`XLConnect`和`openxlsx`包

四、数据集的标注

1、变量标签

R处理变量标签的能力有限，一种解决方法是将变量标签作为变量名，然后通过位置下标来访问这个变量

```
names(patientdata)[2]<-"Age at hospitalization(in years)"
```

说明：使用patientdata[2]来引用这个变量

2、值标签：函数factor()可为分类变量创建值标签

```
patientdata$gender<-factor(patientdata$gender,  
                             levels=c(1,2),  
                             labels=c("male","female"))
```

说明

- levels代表变量的实际值
- labels表示包含了理想值标签的字符型向量

五、处理数据对象的实用函数

1、length()：显示对象中元素/成分的数量

2、dim()、nrow()、ncol()：适用于矩阵、数组、数据框等有维度的对象

3、str()：显示某个对象的结构

4、summary()

5、class()：显示某个对象的类或类型

6、names()、rownames()、colnames()：用于查看名称信息

7、mode()：显示某个对象的模式

8、cbind()、rbind()：按列/行合并对象

9、head()、tail()：列出某个对象的开始或最后部分

10、ls()：显示当前的对象列表

11、rm()：删除一个或多个对象

12、newobject<-edit()：编辑对象并另存为新对象

13、fix()：直接编辑对象

六、小结

- R提供了用于存储数据的各种对象
- 将外源的数据导入R数据框

- 大量的函数用于描述、修改和合并数据结构